Machine-Readable Representation of the Application Example

This section provides a machine-readable representation based on a graph data based on NeoJ4 [10]. The scripts are written in Cypher[4] to model the application example introduced in the main text (cf. Fig. **??** and 2) and further described in appendix 3. Supplying the example as an importable graph dataset enables readers to reconstruct the exact graph used in the article and to explore it with example queries (see below) as well as with their own queries, algorithms, and tools. In the following, we briefly describe how to import the provided Cypher script into a Neo4J[5] database and outline example queries that can be used as a starting point for further explorations of the application example and the method describes in this paper.

The file `setup_graph_db.cypher` (provided as supplementary material) contains the complete set of CREATE statements required to instantiate the graph in a graph database which supports Cypher. Executing the script yields a graph database state that matches the structure of Fig. 2. The script has been imported into and tested with Neo4J version 2025.10.1.[6]

8.5 Neo4J Installation and Data Import

This section briefly outlines the steps required to load the example graph into a Neo4J database. For more details and alternative installation options, the official Neo4j documentation[7] should be consulted.

1. The easiest installation and usage option is provided by installing "Neo4J Desktop". Follow the installation instructions on `https://neo4j.com/docs/desktop/current/installation/`.
2. Open Neo4J Desktop and create a new local database instance. This step requires providing a database name (e.g. `pqc_mig`), a username (default: `neo4j`) and a corresponding password.
3. Start the database instance by clicking on the play button symbol.
4. Connect to the database instance by clicking on the Connect-pulldown-menu and selecting the menu item "query".
5. Paste the contents of the file `setup_graph_db.cypher` into the query field and click on the play button symbol next to the query field (`neo4j$`).
6. This should result in the message "Created 65 nodes, created 190 relationships, set 65 properties, added 65 labels".

After successful execution, the database contains the full application example as used in this article.

[4] `https://neo4j.com/docs/cypher-manual/current/`
[5] `https://neo4j.com/product/neo4j-graph-database/`
[6] `https://neo4j.com/release-notes/database/neo4j-2025-10-1/`
[7] `https://neo4j.com/docs/`

```
MATCH (n)-[r]-(m)
RETURN n, r, m;
```

Listing 1.1. display all nodes and edges

```
MATCH (c:Container)-[:contains]-(n:Node)
WHERE c.name = 'TCU'
AND n.name = 'Secure SW Update'
MATCH (n:Node)-[r]-(m:Node)
RETURN n, r, m;
```

Listing 1.2. display direct connections to one node

8.6 APOC Installation

Some of the below-mentioned queries require the Neo4J library APOC to be installed. Follow the instructions on `https://neo4j.com/docs/apoc/current/installation/` for this purpose and restart the database instance.

8.7 Example Queries

The remainder of this appendix presents a set of example Cypher queries that can be executed on the imported graph. These queries illustrate how the dataset can be explored, how the structural properties described in the main text can be inspected, and how the example can serve as a basis for further experiments. For ease of use (e.g., copy-and-paste into the Neo4j Browser), all example queries presented in this appendix are also provided in the accompanying file `example_queries.cypher`.

The example queries can be pasted into the query field and executed by clicking on the play button symbol next to the query field. The corresponding result is displayed below the query field. The result is displayed as a graph if the option "Graph" is enabled at the top of the display pane.

Display All Nodes and Edges The query in Listing 1.1 display all contents (nodes, edges) of the database (s. Fig. 3, Fig. 4).

Display All Direct Connections to one Node The query in Listing 1.2 displays all connections (ingoing/outgoing) to one specific node (s. Fig. 5). The specific component and node can be filtered by specifying `c.name` and `n.name`. Clicking on one of the displayed neighboring nodes expands the graph further.

Display Redundant Explicit Dependencies The query in Listing 1.3 displays redundant explicit dependencies, which are redundant in the sense that the graph can be (further) transitively reduced by getting rid of these dependencies (s. Fig. 6).

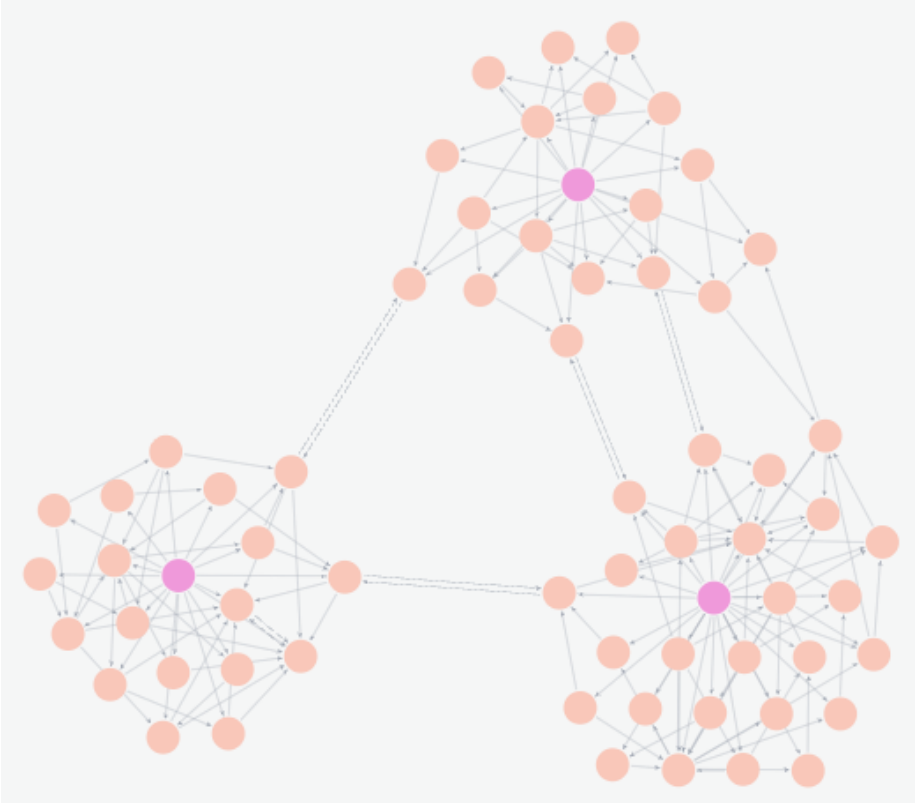

Fig. 3. all nodes and edges

Display Implicit Dependencies The query in Listing 1.4 displays all implicit dependencies which are based on a secure access dependency (s. Fig. 7). The query in Listing 1.5 displays all implicit dependencies (s. Fig. 8).

Display Cycles These queries require the APOC library. The query in Listing 1.6 displays all cycles in the graph (s. Fig. 9). The query in Listing 1.7 displays all cycles and their relationships to the components that contain them (s. Fig. 10).

Display All Edge Types The query in Listing 1.8 displays all edge types in the database (s. Fig. 11).

Clear Database The query in Listing 1.9 permanently deletes all database contents.

```
MATCH (n:Node)-[r:explicit]->(m:Node)
WHERE EXISTS {
  MATCH path = (n)-[:explicit*2..]->(m)
  WHERE NONE(rel IN relationships(path) where rel = r)
}
RETURN n, r, m;
```

Listing 1.3. display redundant explicit dependencies

```
MATCH (n:Node)-[r:secure_access]->(m:Node)
WHERE NOT EXISTS {
  MATCH (n:Node)-[:explicit*1..]->(m:Node)
}
RETURN n, r, m;
```

Listing 1.4. display all implicit dependencies based on functional key dependencies

```
MATCH (n:Node)-[r:shared_keys|secure_boot|secure_access|
    communication_dependency|functional_dependency]->(m:Node)
WHERE NOT EXISTS {
  MATCH (n:Node)-[:explicit*1..]->(m:Node)
}
RETURN n, r, m;
```

Listing 1.5. display all implicit dependencies

```
MATCH (n:Node)
WITH collect(n) as nodes
CALL apoc.nodes.cycles(nodes)
YIELD path RETURN path;
```

Listing 1.6. display all cycles

```
MATCH (n:Node)
WITH collect(n) AS nodes
CALL apoc.nodes.cycles(nodes)
YIELD path
WITH path, nodes(path) AS cycleNodes
MATCH (m:Node)-[r]-(c:Container)
WHERE m IN cycleNodes
RETURN path, m, r, c;
```

Listing 1.7. display all cycles + dependency between nodes and the three container components (ECU; TCU; Server) (zoomed-in)

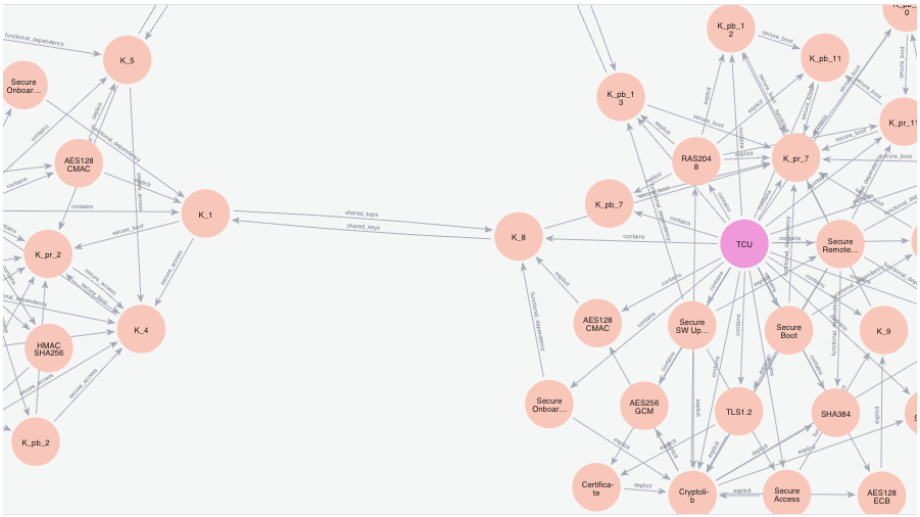

Fig. 4. all nodes and edges (zoomed-in)

```
MATCH ()-[r]->()
RETURN DISTINCT type(r) AS relType
ORDER BY relType;
```

Listing 1.8. all edge types

```
MATCH (n)
DETACH DELETE n;
```

Listing 1.9. clear database

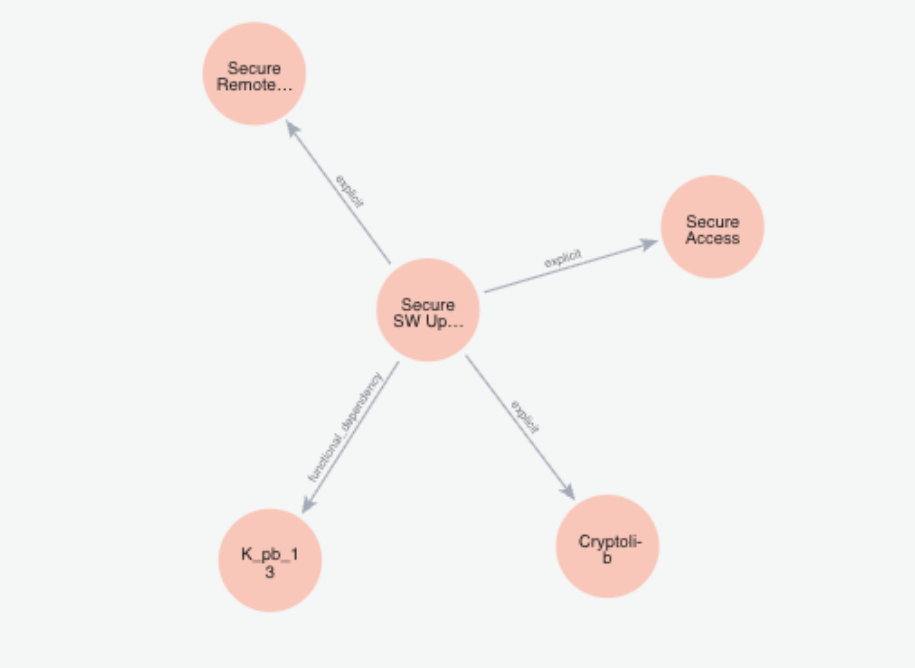

Fig. 5. connections to one node

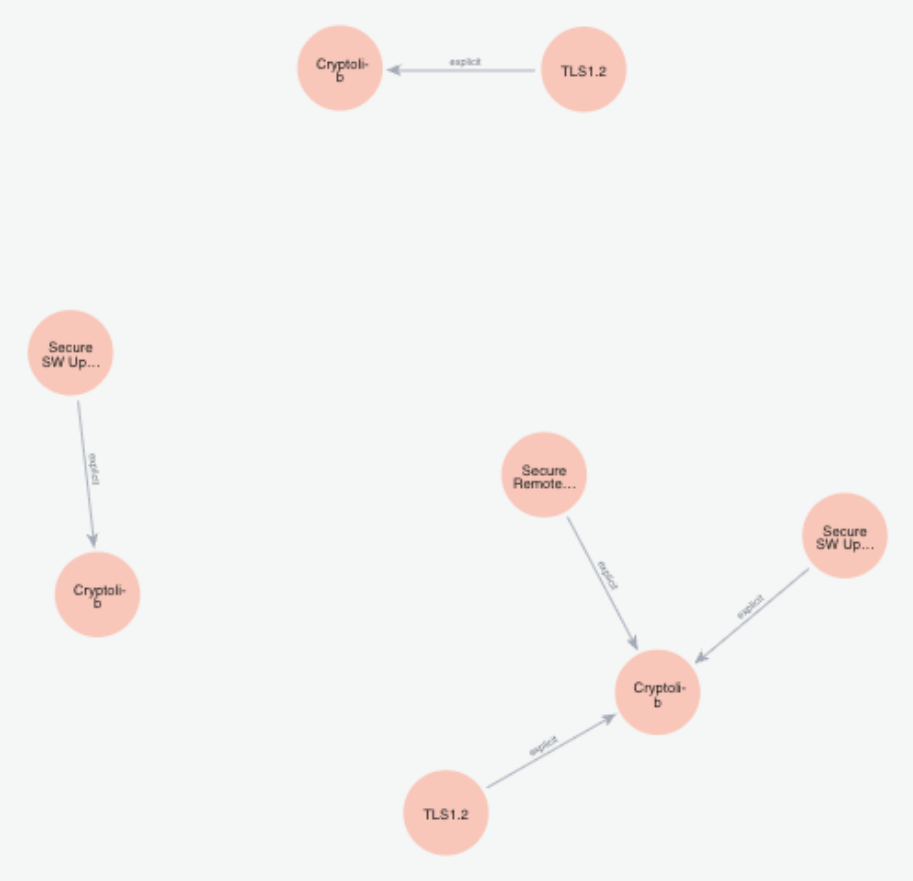

Fig. 6. redundant explicit dependencies

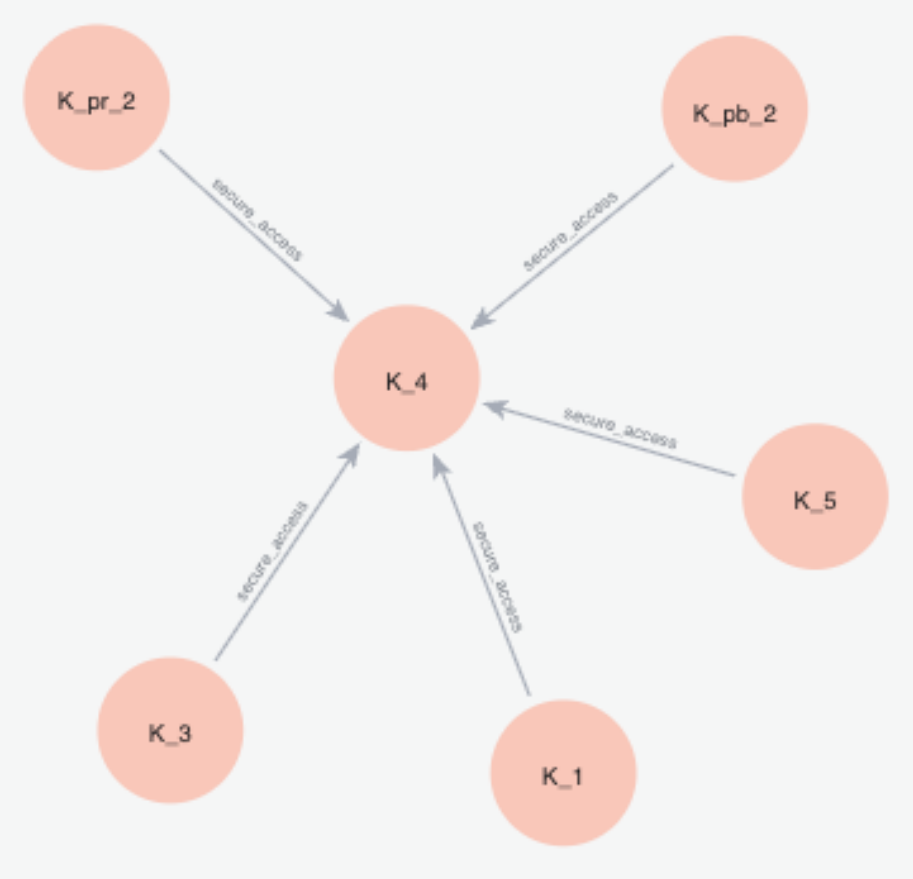

Fig. 7. implicit dependencies (secure access)

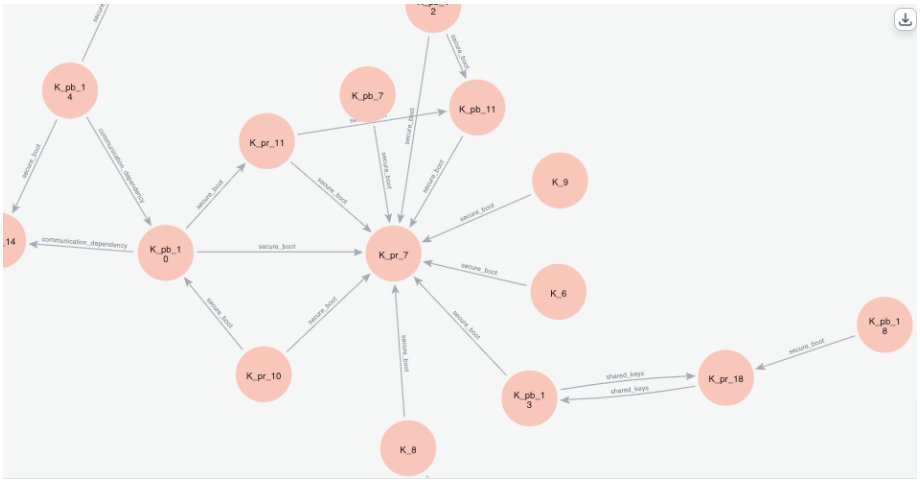

Fig. 8. implicit dependencies (zoomed-in)

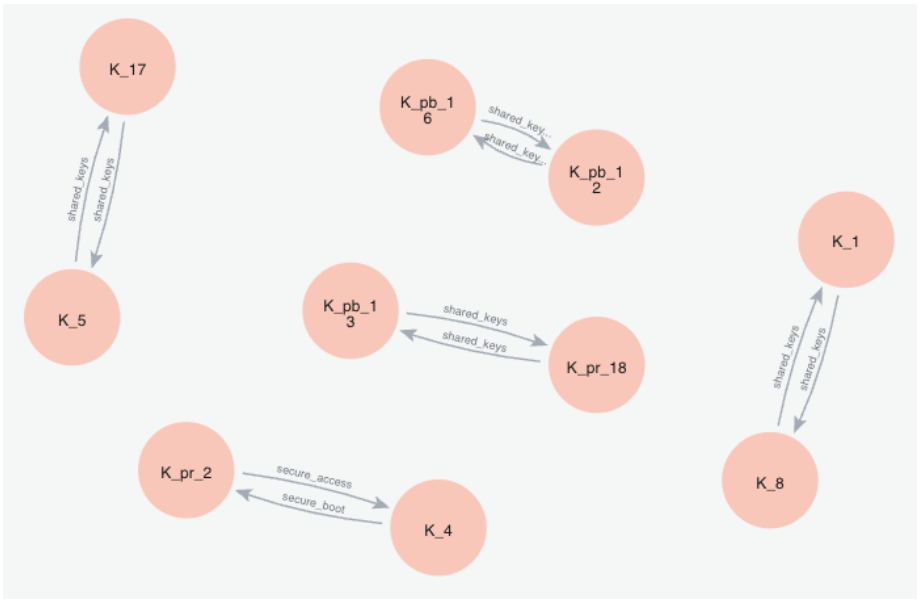

Fig. 9. all cycles

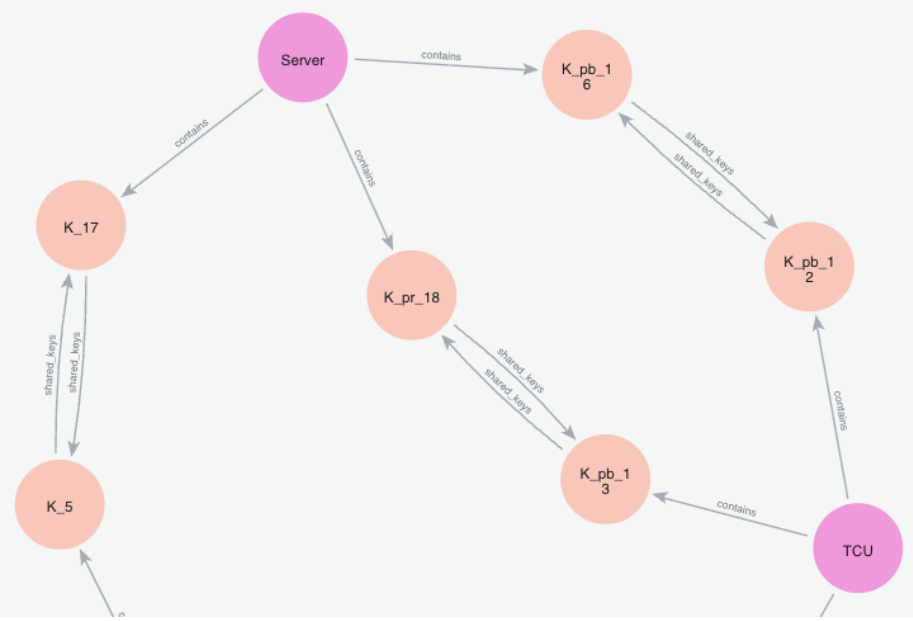

Fig. 10. all cycles with containing components

Table RAW

relType
1 "communication_dependency"
2 "contains"
3 "explicit"
4 "functional_dependency"
5 "secure_access"
6 "secure_boot"
7 "shared_keys"

Fig. 11. all edge types